Design And Analysis of Algorithms
Lecture 2: Deterministic MST algorithms.

January 23, 2018

1 Minimum Spanning Trees, a brief history

Given a graph G with m edges E, n nodes, and a weight function $w : E \to \mathbb{R}$, how quickly can one compute an MST? Here's a brief history of the problem's known time complexity.

1926 Boruvka, "first" algorithm for MSTs, runtime $O(m \ln n)$

1956 Kruskal, same runtime

1930 Jarknik, same runtime

1959 Prim "rediscovered", same runtime

1975 Yao (improved to $m \ln \ln n$).

1984 Freedman and Tarjan mln^*n

1986 Gabor, Galil, Spencer, Tarjan $m \ln \ln^* n$

" Karger Klein, Tarjan $O(m)$ randomized

1997 Chazelle $O(m\alpha(n))$ where α is the inverse Ackerman Function.

1998 Pettie, Ramachandran (optimal algorithm with unknown runtime).

2 MST algorithms

Assume for the remainder of this lecture that all edge weights are unique. This doesn't substantially simplify the algorithmic questions but it does ensure there is a unique MST, and therefore makes it easier to talk about what must be true of **the** MST of a graph G.

The remainder of our work will rely on the following two basic rules regarding MSTs. The first, known as the blue rule, or cut rule, says that the lightest edge across any cut in G must belong to the MST. The second, known as the red rule, or cycle rule, says the heaviest edge in any cycle cannot belong to the MST of G. Let T^* denote the MST of G.

Definition 1 (Cut Rule)**.** *For every cut $(X, N \setminus X)$ in G, the lightest edge across $(X, N \setminus X)$ belongs to T^*.*

Definition 2 (Cycle Rule)**.** *For every cycle $C \subseteq E$ in G, the heaviest edge in C doesn't belong to T^*.*

Algorithm 1: Kruskal's algorithm

Sort edges from lightest to heaviest;
In that order add any edge which reduces the number of connected components to T.

3 Classic algorithms for MSTs

3.1 Kruskal's Algorithm

We now present Kruskal's classic algorithm for finding an MST. Its correctness relies on the previously defined red and blue rules.

To implement this algorithm, one can use a disjoint set union-find datastructure (which maintains the connected component structure of the intermediate forest as the algorithm progresses), which supports the following operations:

- `makeset(elem)`
- `find(elem)`
- `union(elem1, elem2)`

and can do any m of these operations in amortized $O(m\alpha(m))$ time which is dominated by the $O(m \ln m)$ sorting time.

3.2 Prim's Algorithm

Using a slightly different (and more elementary) datastructure, Prim's algorithm achieves the same asymptotic complexity.

Algorithm 2: Prim's algorithm

```
Pick an arbitrary "root" node r ∈ N;
while There is more than one connected component do
    Pick e, cheapest edge from T to N \ T;
    Add e to T;
end
```

This algorithm has at most n iterations of its while loop; if one uses a priority queue implemented as a binary heap to keep track of the lightest $e \notin T$, which has the following operations:

- `insert(elem, key)`
- `decreasekey(elem, key)`
- `extractmin()`$\rightarrow$ `(elem, key)`

where one initially inserts each node into the tree with key ∞ and a dummy edge, then in each round calls `extractmin` and adds the node n to the tree, and decreases the key of all of n's neighbors to $w(v, v')$, there will be n insertions, at most n extractions, and at most m `decreasekey` calls since each call corresponds to an edge in the graph G. This again has $O(m \ln n)$ runtime.

Algorithm 3: Boruvka's algorithm

```
Pick lightest edge from each v ; ;        // If this creates a cycle, drop the heaviest edge
Contract each tree in the forest;
Recurse;
```

4 Boruvka's Algorithm

Boruvka's algorithm doesn't require any fancy datastructures, and has the same runtime as the previous algorithms.

We will show in homework # 1 that each contraction takes $O(m)$ work. Each round at least halves the number of nodes (since each node is adjacent to some edge and we contract along every edge). So, there are $O(\log_2(n))$ contractions, and each round has $O(m)$ work, with $O(m \log(n))$ work in total.

4.1 Improving Prim's Algorithm

One way to improve upon Prim is to use a better datastructure. If we use a Fibonacci heap (of Freedman and Tarjan), this datastructure supports

- `insert(elem, key)`, $O(1)$ amortized
- `decreasekey(elem, key)`, $O(1)$ amortized
- `extractmin()`$\rightarrow$ `(elem, key)`, $O(\ln H)$ amortized, where H upper-bounds the number of elements in the heap.

This alone gives us $O(m+n+n\ln n)$ runtime, and for $m \geq n \ln n$ this is linear. However, we'd ideally like linear (or near linear) in m runtime even when m is of order n. But, this gives us a method for improving upon Prim in a more significant way: if we could manage to run Prim and guarantee that there were never too many nodes in the heap, we could possibly improve upon this runtime.

4.2 Freedman and Tarjan's $O(m \ln^* n)$ algorithm

How should we try and modify Prim (implemented with a Fibonacci heap, therefore with runtime $O(m + n + n \ln H)$) to reduce the $O(n \ln n)$ term? If we could somehow guarantee that the number of elements in the heap was upper-bounded by something less than n, that could help. So, rather than keeping every vertex in the heap, why don't we simply keep those vertices immediately adjacent to the current tree in the heap? E.g., if the heap only contains the boundary vertices between T and G, and if that were guaranteed to be small, we could be done.

So, if the boundary of T and $G \setminus T$ ever gets large, we'll simply contract T into one node and restart Prim on the contracted graph. This will guarantee that Prim (if restarted at this node) would have boundary size at most 1, so there will always be a way to restart Prim such that it has small boundary.

Formally, the "improved" contraction Prim will work as follows. It keeps the heap to be size at most K for any K, and contracts the current tree kept by Prim anytime the boundary gets too large.

Algorithm 4: Freedman and Tarjan's subroutine, parameterized by K

```
Pick an unmarked vertex v, start Prim(v) making T;
for each round of Prim(v) do
    Add (u, min_{v∈T} w(u,v)) to heap ∀u ∈ N(T) = {u : (uv) ∈ E, v ∈ T, u ∉ T};
    if |heap| > K or T adds an edge to a marked vertex u then
        mark all of T's vertices, restart prim above
    end
end
Stop when every vertex is added to a tree or when Prim adds a marked vertex to T.
```

This subroutine "contracts" the current tree by marking its vertices. Its runtime (between recursions) is $O(m + n\ln(K))$: Prim does $O(m+n)$ work on the insertions and decreases, and since each round of Prim reduces the number of connected components by at least 1 and uses only one extractmin there are at most $O(n)$ calls to `extractmin`, and the heap never is larger than K by construction. After each run of Prim, we have a forest whose edges all belong to T^* by the cut rule.

The question then is how do we set K? Note that any marked vertex u belongs to a connected component C such that $\sum_{v\in C} d_v \geq K$: if it was marked due to its tree having large neighborhood then this is clearly true; if it was marked because T just added a marked node v to T, then v had the property that its component C' was such that $\sum'_{v'\in C} d_{v'} \geq K$, and adding u to this component only increases this sum. So, for the final components $C_1, \ldots, C_\ell$, we have that

$$2m = \sum_v d_v = \sum_{i=1}^{\ell} \sum_{v\in C_i} d_v \geq \ell K$$

so $2m/K \geq \ell$; this routine does not produce very many trees.

In entirety, the algorithm will therefore have $n_i \leq n$ and $m_i \leq m$ nodes and edges in the ith run of the subroutine, with runtime $O(m_i + n_i \ln K_i)$ and the question is how to set K_i such that $\sum_i O(m_i + n_i \ln K_i)$ is minimized. If $K_i = 2^{\frac{2m}{n_i}}$, then the work in run i is $O(m_i + m) = O(m)$. How many rounds i do we have in this scheme? n_{i+1}, the number of nodes we have in round $i+1$, is simply the number of trees we had in round i, which we proved above is at most $\frac{2m_i}{K_i}$, thus

$$n_{i+1} \leq \frac{2m_i}{K_i}$$

and moreover, by definition we have $K_{i+1} = 2^{\frac{2m}{n_{i+1}}}$ or equivalently that $n_{i+1} = \frac{2m}{\ln K_{i+1}}$. Thus, we have that

$$\frac{2m}{\ln K_{i+1}} \leq \frac{2m_i}{K_i} \Rightarrow K_i \leq \ln K_{i+1}$$

and so $K_{i+1} \geq 2^{K_i}$. The threshold value increases tetrationally (exponential in itself) each round, so in $O(\ln^* n)$ rounds it will exceed n. Once K exceeds n, the algorithm will simply run standard Prim. Thus that upper-bounds the number of executions of the subroutine, each of which have linear work for a total runtime of $O(m \ln^* n)$.

References

[1] Bernard Chazelle, *A minimum spanning tree algorithm with inverse-ackermann type complexity*, Journal of the ACM (JACM) **47** (2000), no. 6, 1028–1047.

[2] Michael L Fredman and Robert Endre Tarjan, *Fibonacci heaps and their uses in improved network optimization algorithms*, Journal of the ACM (JACM) **34** (1987), no. 3, 596–615.

[3] Harold N Gabow, Zvi Galil, Thomas Spencer, and Robert E Tarjan, *Efficient algorithms for finding minimum spanning trees in undirected and directed graphs*, Combinatorica **6** (1986), no. 2, 109–122.

[4] Vojtěch Jarník, *O jistém problému minimálním.(z dopisu panu o. boruvkovi)* (1930).

[5] David R Karger, Philip N Klein, and Robert E Tarjan, *A randomized linear-time algorithm to find minimum spanning trees*, Journal of the ACM (JACM) **42** (1995), no. 2, 321–328.

[6] Joseph B Kruskal, *On the shortest spanning subtree of a graph and the traveling salesman problem*, Proceedings of the American Mathematical society **7** (1956), no. 1, 48–50.

[7] Seth Pettie and Vijaya Ramachandran, *An optimal minimum spanning tree algorithm*, Automata, Languages and Programming (2000), 49–60.

[8] Andrew Chi-Chih Yao, *An 0 (— e— loglog— v—) algorithm for finding minimum spanning trees*, Information Processing Letters **4** (1975), no. 1, 21–23.